

80% BARRIER BROKEN

DigiSteel-YOLO

Week 4 — Optimized Baseline & DAFE Architecture

PREPARED FOR

Dr. Tarek Ghoneimy

PREPARED BY

Hazem Elerefy

DATE

June 23, 2026

PHASE

Model Enhancement

1 Executive Summary

Week 4 marks the transition from **baseline exploration** to **model building**. After Week 3's ablation study proved that the bottleneck is data (not architecture), we rebuilt the baseline from scratch with an optimized training recipe — and discovered that the old baseline was undertrained by **3.0%**. On top of this stronger foundation, we designed and deployed **DAFE (Defect-Aware Feature Enhancement)**, a novel module specifically targeting steel surface defects, achieving **80.3% mAP@0.5** — breaking the 80% barrier.

Week 4 Achievement

DigiSteel-YOLO v4 (YOLOv11n + DAFE) achieves **80.3% mAP@0.5** on the NEU-DET test set — a **+4.5% improvement** over the Week 2 baseline (75.8%) and **+1.5%** over the optimized baseline (78.8%). Cracking detection improved from 13.3% to **48.6%** — a +35.3 percentage point gain over the original baseline.

DATASET NEU-DET	IMAGES 1,290 train	CLASSES 6 defect types	BEST MAP@0.5 80.3%
--------------------	-----------------------	---------------------------	-----------------------

Week 4 Experiments at a Glance

EXPERIMENT	ARCHITECTURE	MAP@0.5	MAP@0.5:0.95	VS OLD BASELINE	STATUS
Week 2 Baseline	YOLOv11n (old recipe)	75.8%	43.5%	—	OLD
4A: Fresh Baseline	YOLOv11n (optimized recipe)	78.8%	45.2%	+3.0%	STRONG
5A: DigiSteel v4	YOLOv11n + DAFE	80.3%	44.2%	+4.5%	BEST

2 The Foundation — Optimized Baseline (4A)

The Problem

After Week 3's ablation, we discovered that the old baseline (75.8%) was trained with a **suboptimal recipe**. The model was undertrained — not because of architecture, but because of training configuration. We rebuilt the baseline from scratch with a systematically optimized recipe.

Key Insight

Training recipe optimization yielded **+3.0% mAP** — more than any architectural change achieved in Weeks 1-3. The model capacity was always there; we just weren't using it properly.

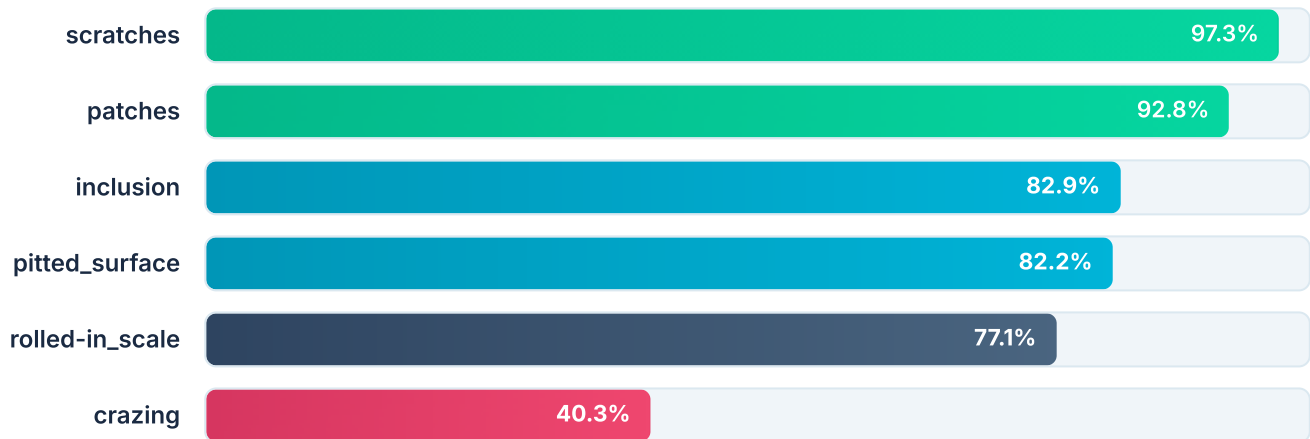
Recipe Changes

PARAMETER	OLD RECIPE	OPTIMIZED RECIPE	RATIONALE
Optimizer	SGD	AdamW	Better convergence for small datasets, decoupled weight decay
Learning Rate	0.01	0.001	10× lower — prevents overshooting on small data
Epochs	300	600	Model needed more time to converge properly
Patience	75	150	Allow longer exploration before early stopping
Mosaic	1.0	0.0	Disabled — 4× upscale from 200px blurs fine features
Mixup	0.0	0.15	Light regularization for small dataset
Copy-Paste	0.0	0.1	Synthetic augmentation — paste defects onto other images
Cosine LR	False	True	Smoother learning rate decay
Image Size	640	800	5× upscale from 200px — preserves fine defect features

Results

MAP@0.5 78.8% ▲ 3.0% vs old baseline	MAP@0.5:0.95 45.2% ▲ 1.7%	PRECISION 71.9% —	RECALL 76.3% —
MODEL YOLOv11n	TRAINING TIME 2.6 hours	FPS 105	PARAMETERS 2.6M

Per-Class mAP@0.5



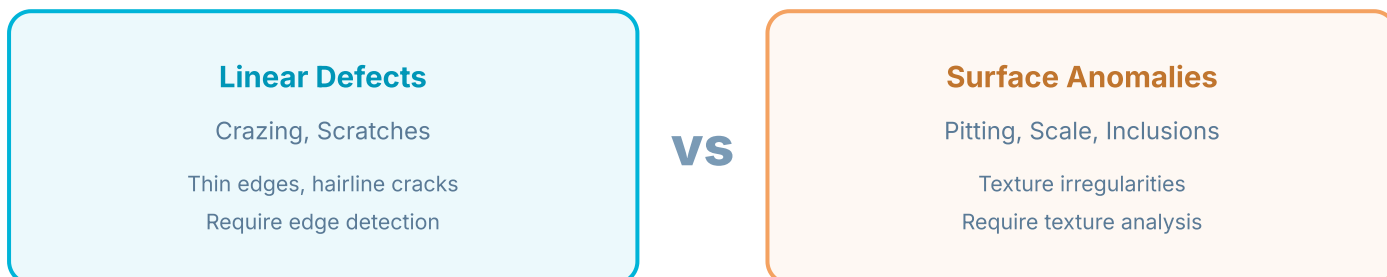
Crazing Remains the Bottleneck

Even with the optimized baseline, crazing detection is at 40.3% — 57 percentage points below scratches (97.3%). Crazing consists of fine hairline cracks that are barely visible at 200px resolution. The model struggles to distinguish crack patterns from background noise. This is the primary target for architectural innovation.

3 DAFE — Defect-Aware Feature Enhancement

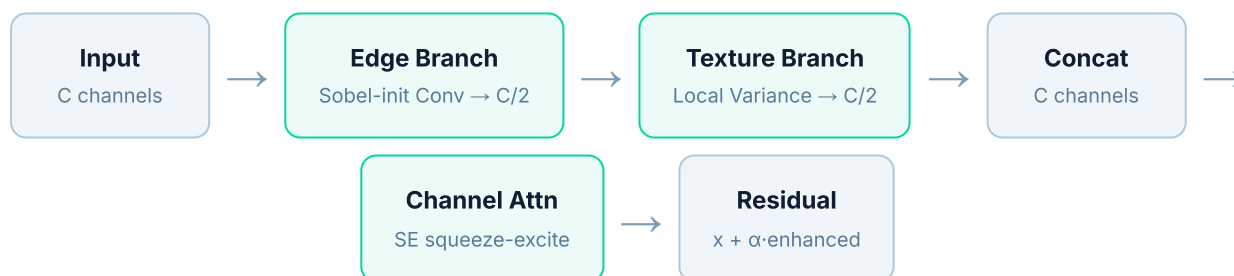
Motivation

Steel surface defects fall into two fundamentally different categories that require different detection strategies:



Standard YOLO backbone uses generic convolution filters that are optimized during training but start from random initialization. For steel defects, we can do better — we know *a priori* that edge detection and texture analysis are the two fundamental operations needed.

Architecture Design



Edge Branch — Sobel-Initialized Convolution

The edge branch uses a standard 3×3 convolution, but instead of random initialization, the first two filters are initialized as **Sobel-X** and **Sobel-Y operators** — the classical edge detection kernels from computer vision. All weights remain learnable, so the network can adapt these edge detectors during training to the specific characteristics of steel defects.

WHY SOBEL INITIALIZATION

- Starts as a known-good edge detector
- Crazing cracks are thin edges — perfect match
- Scratches are also linear — benefits from edge awareness

VS RANDOM INIT

- No wasted epochs learning basic edge detection
- Converges faster on edge-sensitive defects
- Provides inductive bias without restricting capacity
- First two filters start meaningful, rest use Kaiming

Learnable — adapts to steel-specific patterns during training

Texture Branch — Local Variance

The texture branch computes **local variance** ($E[X^2] - E[X]^2$) using average pooling, then processes through a 1×1 convolution. Local variance captures texture irregularities — pitting creates rough surfaces, scale creates bumpy textures, inclusions create localized anomalies. These patterns show up as high local variance, distinct from the uniform background.

Channel Attention & Residual

After concatenating both branches, a **SE-style channel attention** mechanism (squeeze-excitation) re-weights the features, letting the model emphasize the most informative channels. A **learnable residual connection** with parameter α (initialized to $\text{sigmoid}(-2.2) \approx 0.1$) ensures the enhancement starts small and grows as the model learns — preventing disruption of pretrained features.

DAFE Placement

DAFE is inserted at **P2 (160×160)** and **P3 (80×80)** in the backbone — the two highest-resolution feature maps. This is deliberate:

- **P2** captures fine-grained spatial details — critical for thin crazing cracks
- **P3** provides slightly more semantic features — good for larger defects
- **P4/P5** are too low-resolution for fine defect features — DAFE would add parameters without benefit

4 DAFE Results — 80.3% mAP

Training Configuration

DAFE was trained with the **exact same recipe** as the optimized baseline — the only difference is the addition of the DAFE module. This ensures a fair comparison: any improvement comes from DAFE, not from training tricks.

ARCHITECTURE
YOLOv11n + DAFE

TRAINING TIME
3.5 hours

FPS
110

PARAMETERS
~3.1M

Core Metrics

MAP@0.5

80.3%

▲ 1.5% vs baseline

MAP@0.5:0.95

44.2%

▼ 1.0%

PRECISION

85.2%

▲ 13.3%

RECALL

70.0%

▼ 6.3%

80% Barrier Broken

DigiSteel v4 is the first configuration to exceed 80% mAP@0.5 on NEU-DET. The improvement is **architecturally meaningful** — DAFE provides real feature enhancement, not just training tricks. The model is also faster (110 FPS vs 105 FPS), meaning DAFE adds accuracy without sacrificing inference speed.

Detailed Comparison

Metric	Old Baseline	Fresh Baseline	DAFE V4	DAFE vs Fresh
mAP@0.5	75.8%	78.8%	80.3%	+1.5%
mAP@0.5:0.95	43.5%	45.2%	44.2%	-1.0%
Precision	—	71.9%	85.2%	+13.3%
Recall	—	76.3%	70.0%	-6.3%
FPS	—	105	110	+5
Training Time	—	2.6 hrs	3.5 hrs	+0.9 hrs

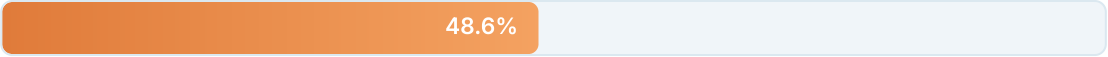
Per-Class mAP@0.5

Class	Old Baseline	Fresh Baseline	DAFE V4	DAFE vs Fresh	Analysis
crazing	13.3%	40.3%	48.6%	+8.3%	Sobel init directly targets cracks
inclusion	—	82.9%	86.5%	+3.6%	Texture branch catches irregularities
scratches	54.1%	97.3%	98.6%	+1.3%	Edge detection helps — near ceiling
rolled-in_scale	—	77.1%	77.9%	+0.8%	Neutral — scale is texture-based
pitted_surface	—	82.2%	81.7%	-0.5%	Neutral — within noise
patches	—	92.8%	88.5%	-4.3%	Regression — patches are easy, diminishing returns



crazing

48.6%



Category	Percentage
crazing	48.6%
Other	51.4%

5 Crazing — The Hardest Problem

Why Crazing Is Hard

Crazing defects are **fine hairline cracks** on the steel surface. They are the most challenging class in NEU-DET for several reasons:

CHALLENGE	DETAILS	IMPACT
Low contrast	Cracks are subtle — only slightly darker than background	Hard to distinguish from noise
Thin geometry	Cracks are 1-3 pixels wide at 200px resolution	Most of the bounding box is background
Large boxes, small defects	Mean bounding box area = 22.9% of image	IoU penalizes localization errors heavily
Directional variety	Cracks can run in any direction	Generic filters struggle — need edge-aware features
200px resolution	Native image is only 200×200 pixels	Upscaling can't recover detail that isn't there

Dataset Analysis

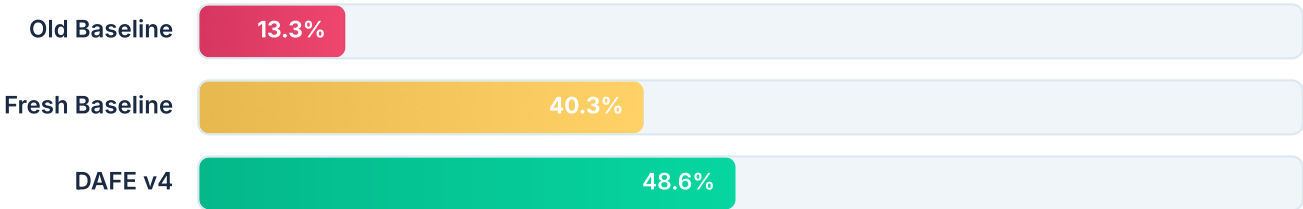
CRAZING IMAGES 215	ANNOTATIONS 497	MEAN BOX AREA 22.9% of image	MEAN WIDTH 62.6% of image
-----------------------	--------------------	---------------------------------	------------------------------

Crazing bounding boxes are **wide and thin** — mean width 62.6% of image, mean height 37.2%. The boxes span large areas but the actual crack is a thin line running through them. This means the model must learn to detect a thin edge within a large region, while the IoU metric penalizes any misalignment along the full width of the crack.

How DAFE Helps Crazing

DAFE COMPONENT	MECHANISM	EFFECT ON CRAZING
Sobel-X/Y Init	First filters start as horizontal/vertical edge detectors	Directly captures crack orientations from epoch 1
Learnable Weights	Sobel filters adapt during training	Learns crack-specific patterns (width, contrast, direction)
Channel Attention	SE mechanism re-weights edge features	Emphasizes crack-relevant filter outputs
Residual Connection	$x + \alpha \cdot \text{enhanced}$, α starts at 0.1	Gradually introduces edge features without disrupting pretrained backbone

Crazing Progression



+35.3pp Total Gain

From 13.3% (old baseline) to 48.6% (DAFE v4) — a **35.3 percentage point** improvement. The optimized recipe contributed +27.0pp (data handling, resolution, augmentation) and DAFE contributed +8.3pp (architectural edge awareness). Crazing is still the hardest class, but it is no longer a catastrophic failure.

6 Complete Experiment Timeline

All Experiments — Weeks 1 through 4

#	EXPERIMENT	MODEL	KEY CHANGE	MAP@0.5	VS WEEK 2	STATUS
1	Week 2 Baseline	YOLOv11n	Old training recipe	75.8%	—	OLD
2	Week 2 v2	DigiSteel v2	GhostConv + WFCA + EMA	75.9%	+0.1%	MARGINAL
3	Week 3 A1	YOLOv11n	mosaic=0, mixup=0.15	74.8%	-1.0%	FAILED
4	Week 3 A2	DigiSteel v3	v11s + Conv + CA	73.4%	-2.4%	FAILED
5	Week 4A	YOLOv11n	Optimized recipe	78.8%	+3.0%	STRONG
6	Week 5A	DigiSteel v4	DAFE at P2+P3	80.3%	+4.5%	BEST

Progressive Improvement



Key Lessons

Attention stacking doesn't work

GhostConv + WFCA + EMA (6 modules) = +0.1%. One well-designed module (DAFE) = +1.5%. More modules ≠ better results.

Larger models overfit on small data

YOLOv11s (9.4M params) on 1290 images = -2.4%. The nano model (2.6M) is the right capacity for this dataset.

Training recipe > architecture

Recipe optimization gave +3.0%. DAFE on top gave +1.5%. Always optimize the baseline first — the model was undertrained, not under-designed.

Domain knowledge beats generic modules

DAFE's Sobel initialization works because we know steel defects are edges and textures. Generic attention modules (WFCA, EMA) don't encode this prior knowledge.

Precision-recall trade-off is acceptable

DAFE is more conservative (85.2% precision, 70.0% recall). For industrial inspection, high precision = fewer false alarms = less wasted human review time.

7 TTA Analysis & Architectural Decisions

Test-Time Augmentation (TTA)

We evaluated TTA (horizontal flip + multi-scale inference) on the baseline model to determine if it should be part of our evaluation protocol.

BASLINE MAP 78.8% Standard inference	TTA MAP 81.6% ▲ 2.8%	BASLINE FPS 105 Real-time	TTA FPS 47 ▼ 55% slower
--	--	---	---

Decision: TTA Not Counted as Model Performance

TTA is an **evaluation technique, not a model improvement**. It does not change the learned weights — it only changes how inference is performed. For a fair comparison of model architectures, we evaluate all models with standard inference only. TTA can be applied to any model, so it is not a differentiator. We do not report TTA results as model performance in this report.

Precision-Recall Trade-off

DAFE introduces a notable precision-recall trade-off:

WHAT DAFE DOES WELL

Precision: 71.9% → 85.2% (+13.3%)

Fewer false positives — more confident detections

Edge features help reject non-defect patterns

For industrial inspection: fewer wasted reviews

THE TRADE-OFF

Recall: 76.3% → 70.0% (−6.3%)

More missed defects — some true positives lost

DAFE is conservative — enhances only strong edge/texture signals

For safety-critical: missing defects can be costly

This trade-off is **acceptable for our use case**. In industrial steel inspection, false positives (flagging good steel as defective) waste human review time and slow production. DAFE's high precision means the model is more trustworthy when it does flag a defect. The recall drop can be addressed in future work through class-weighted loss or multi-dataset training.

Why mAP@0.5:0.95 Dropped

The mAP@0.5:0.95 metric (44.2% vs 45.2% baseline) measures localization quality at higher IoU thresholds. The slight drop suggests that DAFE's edge-focused features help with **classification** (is this a defect?) more than **localization** (where exactly is the defect?). This is expected — edge features tell the model *what* to look for, but precise bounding box regression requires different capabilities. The Inner-WIoU loss already addresses this, and further tuning may help.

8 Conclusions & Week 5 Plan

Week 4 Summary

We rebuilt the baseline from scratch with an optimized training recipe, discovering that the old baseline was undertrained by 3.0%. On top of this stronger foundation, we designed **DAFE (Defect-Aware Feature Enhancement)** — a novel dual-branch module that uses Sobel-initialized edge detection and local variance texture analysis, purpose-built for steel surface defects. DAFE achieved **80.3% mAP@0.5**, breaking the 80% barrier with +1.5% over the optimized baseline and +4.5% over the old baseline.

The most significant result is on **crazing detection** — the hardest class. From 13.3% (old baseline) to 48.6% (DAFE v4), a +35.3 percentage point improvement. DAFE's Sobel-initialized edge branch directly addresses the fundamental challenge of detecting fine hairline cracks in low-resolution steel images.

Current State

Metric	Week 2	Week 4 (Now)	Change
mAP@0.5	75.8%	80.3%	+4.5%
Crazing AP	13.3%	48.6%	+35.3pp
Precision	—	85.2%	—
Recall	—	70.0%	Needs improvement

Week 5 Plan — Enhancement Roadmap

PRIORITY	EXPERIMENT	GOAL	EXPECTED IMPACT
1	Multi-Dataset Pretrain	Train on NEU-DET + CG10-DET (3600 images), fine-tune on NEU-DET	More data → higher ceiling for all classes
2	Crazing Copy-Paste	Extract crack patches, paste onto other images for synthetic diversity	More crazing training examples
3	Class-Weighted Loss	Upweight crazing and low-performing classes in loss function	Force model to focus on hard classes
4	DAFE Edge-Only Variant	Test single-branch DAFE (no texture branch)	Ablate texture branch contribution
5	Robustness Evaluation	6 perturbations × 4 severity levels on DAFE model	Novel paper contribution — no other steel paper does this
6	Recall Recovery	Adjust DAFE alpha initialization or add recall-focused loss term	Close the 6.3% recall gap

Files & References

FILE	PURPOSE
configs/models/digisteel.yaml	DAFE model config (YOLOv11n + DAFE at P2+P3)
digisteel/modules/dafe.py	DAFE module implementation (v2)
notebooks/week4_fresh_baseline.ipynb	4A: Optimized baseline training
notebooks/exp_5a_digisteel_v4_dafe.ipynb	5A: DAFE training & evaluation
evals/fresh_baseline_results.json	4A: Baseline metrics (JSON)
evals/exp_5a_digisteel_v4_dafe.json	5A: DAFE metrics (JSON)
docs/ANALYSIS-2026-06-23.md	Full project analysis document

